

[3페이지 — Motivation]

먼저 저희가 4-way Dot Product accelerator 및 MAC 구조를 설계하게 된 배경부터 설명드리겠습니다.

최근 CNN과 같은 신경망에서는 이미지나 특징 데이터를 처리하기 위해 매우 많은 convolution 연산이 반복적으로 수행됩니다. 그런데 convolution 연산을 하드웨어 관점에서 살펴보면, 복잡해 보이는 연산의 상당 부분이 결국 입력 activation과 weight를 곱하고, 그 결과를 누적하는 연산으로 구성되어 있습니다. 즉 같은 위치의 input과 weight를 각각 곱한 다음, 생성된 product들을 모두 더하는 형태입니다.

이러한 연산이 바로 Dot Product이며, CNN의 연산 성능을 높이기 위해서는 이러한 곱셈과 누산을 얼마나 효율적으로 처리하느냐가 중요하다고 생각했고, 이를 실제 transistor-level full-custom 회로로 구현하고 성능을 비교해보는 것을 프로젝트의 출발점으로 설정했습니다.

[7페이지 — Overall Architecture]

보이는 것처럼 두 방식은 같은 계산을 서로 다른 하드웨어 구조로 수행합니다.

[8페이지]

MAC은 하나의 multiplier와 adder를 반복해서 사용하기 때문에 면적 측면, 하드웨어 재사용에는 유리하지만, register와 feedback으로 인해 여러 clock cycle이 필요하기 때문에 연산 시간이 더 오래 걸리게 됩니다.

[9페이지]

반면 Parallel Dot Product Accelerator는 multiplier 4개와 adder tree를 사용하는 병렬 방식으로 하나의 clk 사이클에 4개 계산을 수행할 수 있습니다.

[12페이지]

결론적으로 면적, Delay, Power, Throughput등을 주요 성능 지표로 분석하여 Sequential MAC과 Parallel Dot Product Accelerator가 각각 면적과 연산 속도, 전력 및 에너지 효율 측면에서 어떤 trade-off를 가지는지 정량적으로 비교하는 것을 최종 목표로 계획했습니다.

[15페이지 — Multiplier Design]

4bit MUL에 사용된 컴포넌트는 다음과 같습니다.

[16페이지]

크게는 AND 블록과 ADDER 블록으로 나누어서 배치했습니다.

[17페이지]

먼저 AND block은 4비트 입력 A와 B의 모든 bit 조합에 대해 partial product를 생성하는 역할을 합니다. 1비트 곱셈은 AND 연산과 동일하기 때문에, 예를 들어 A0와 B0를 AND하면 PP00, A1과 B2를 AND하면 PP12와 같은 partial product가 만들어집니다.

A가 4비트이고 B도 4비트이기 때문에 가능한 조합은 총 16개이고, 따라서 AND block에는 총 16개의 AND cell이 사용됩니다.

여기서 중요한 점은 AND block이 최종 곱셈 결과를 만드는 것이 아니라, 곱셈에 필요한 중간 항인 partial product만 생성한다는 점입니다.

전체 partial product는 PP00부터 PP33까지 총 16개가 있고, 각각의 값은 서로 다른 bit weight를 가집니다. 예를 들어 최종 output bit중 하나인 PP00은 가장 낮은 weight를 갖고, PP10과 PP01은 같은 다음 weight에 해당하며, PP20, PP11, PP02는 그 다음 weight에 해당합니다.

따라서 AND block의 출력은 이후 adder network에서 weight별로 합산되어야 하는 입력 데이터라고 볼 수 있습니다.

[18페이지]

다음은 ADDER block입니다. 세로 방향으로 같은 stage의 인스턴스들을 배치했습니다. ADDER block의 역할은 앞선 AND block에서 생성된 16개의 partial product를 bit weight에 맞춰 단계적으로 더해서 최종 8비트 곱셈 결과 P를 만드는 것입니다.

해당 구조에서는 이 부분을 4개의 Half Adder와 8개의 Full Adder, 총 12개의 adder cell로 구성했습니다.

Half Adder는 두 개의 입력을 더해 SUM과 CARRY를 만들고, Full Adder는 두 개의 데이터 입력과 이전 단계에서 전달된 CIN까지 총 세 개의 값을 더합니다.

화면에 보이시는 것처럼 이렇게 carry는 같은 stage의 다음 adder로 전달되고, sum은 다음 stage의 adder로 전달되는 구조입니다.

이런 방식으로 각 stage에서 SUM은 해당 bit의 결과 또는 다음 단계의 입력으로 사용되고, CARRY는 한 단계 높은 weight로 전달됩니다.

[19페이지 — RCA10 Design]

다음으로 RCA10에 대해 설명드리겠습니다.

RCA10은 10개의 Full Adder를 직렬로 연결한 10-bit Ripple Carry Adder입니다. MAC에서는 MUL4가 생성한 8-

bit product를 zero extension하여 10-bit로 만든 뒤, 이전 누적값인 레지스터 출력 Q와 더하는 역할을 합니다.

[20페이지]

각 Full Adder에서 발생한 COUT은 다음 bit Full Adder의 CIN으로 전달됩니다. 따라서 carry가 LSB에서 MSB 방향으로 순차적으로 전파되기 때문에 Ripple Carry Adder라고 합니다.

RCA10의 10bit output SUM은 REG10에 저장되고, REG10의 Q는 다시 RCA10 입력으로 feedback됩니다. 이를 통해 매 clock마다 기존 누적값에 새로운 MUL4 결과를 더하는 MAC 연산을 수행합니다.

하나의 사이클동안의 계산은 8bit도 가능하지만, 각 사이클의 최대 결과는 225이고, 이를 네 번 누적하면 최대 900이므로 최종 누적값을 표현하기 위해 10-bit RCA와 10-bit register를 사용했습니다.

[21페이지 — Register / DFF Reset]

다음으로 설계한 Register design 및 Reset 구조에 대해 설명드리겠습니다.

Register의 초기 상태를 안정적으로 0으로 설정하기 위해 DFF 출력단에 NMOS를 하나 추가해 reset 기능을 구현했습니다.

[22페이지]

기존 DFF는 별도의 초기화 경로가 없기 때문에 transient simulation 시작 시 Q 값이 불확정 상태가 될 수 있습니다. 특히 본 설계의 MAC은 register 출력이 다시 RCA 입력으로 feedback되는 구조이기 때문에 초기 누산값이 0이 아니면 이후 모든 연산 결과에 오차가 누적됩니다.

[23페이지]

이를 해결하기 위해 Q node와 VSS 사이에 NMOS를 추가하고, NMOS의 gate를 RSTB 단자로 설정하였습니다.

1V가 인가되어서 Reset 신호가 활성화되면 NMOS가 ON되어 DFF output인 Q를 VSS, 즉 logic 0로 강제로 pull-down합니다. Reset에 0V, 즉 reset이 해제되면 NMOS는 OFF되어 기존 DFF의 정상적인 데이터 저장 동작에 영향을 주지 않습니다.

이를 통해 clock과 관계없이 Q를 초기화할 수 있기 때문에 10bit 레지스터에서는 10개의 DFF 각각에 동일한 reset 신호를 적용하여 전체 accumulator를 0000000000으로 초기화할 수 있었습니다.

[24페이지]

사진에서 보시는 것처럼 시뮬레이션 시에 초기에 짧은 시간 동안 리셋 단자에 1V의 신호를 주어서, 레지스터의 초기값을 모두 0으로 초기화하여 이후 누적되는 계산에 오류가 없도록 하였습니다.

[25페이지 — MAC Schematic]

앞서 만든 인스턴스들을 이용해 MAC 구조를 설계하였습니다.

[26페이지]

전체 스키매틱은 다음과 같습니다. MUL를 통해서 두 input의 곱을 계산하고, 레지스터는 이전에 계산된 값을 RCA로 피드백해서 새로 계산된 결과와 더해지도록 설계하였습니다.

[27페이지]

다음은 RCA10의 비트 수를 10비트로 설정한 이유와 상위 2비트를 VSS에 연결한 이유입니다.

먼저 4비트 unsigned multiplier의 경우 각 입력의 최댓값은 15이므로, 한 번의 곱셈 결과는 최대 15 곱하기 15인 225가 됩니다.

4way Dot Product 연산에서는 이러한 곱셈 결과 4개를 최종적으로 더하기 때문에, 전체 결과의 최댓값은 225 곱하기 4인 900이 됩니다.

9비트로 표현할 수 있는 최댓값은 511이기 때문에 900을 표현하기에는 부족하고, 10비트에서는 최대 1023까지 표현할 수 있습니다. 따라서 최종 연산 결과를 overflow 없이 표현하기 위해 최소 10비트의 RCA가 필요합니다.

하지만 MUL4에서 나오는 하나의 곱셈 결과 자체는 8비트이기 때문에, 10비트 RCA에 연결하면 상위 2비트가 남게 됩니다.

이 두 비트를 연결하지 않고 두면 floating 상태가 되어 데이터값이 명확한 0이나 1로 결정되지 않을 수 있고, 시뮬레이션이나 실제 회로 동작에서 불안정한 입력으로 작용할 수 있습니다.

따라서 상위 2비트를 VSS에 연결하여 확실한 Logic 0으로 고정했습니다. unsigned 데이터이기 때문에 상위에 0을 추가하는 zero extension을 적용해도 기존 8비트 곱셈 결과의 값은 변하지 않습니다. 이러한 방식으로 8비트 MUL4 출력을 RCA10에 입력할 때 10비트 데이터로 확장했습니다.

[28페이지]

다음은 Parallel Dot Product Accelerator의 설계입니다.

Sequential MAC이 하나의 multiplier와 RCA를 여러 clock에 걸쳐 재사용하는 것과 달리, Dot Product 구조에서는 4개의 MUL4를 병렬로 배치하여 4개의 곱셈을 동시에 수행하도록 구성했습니다.

[29페이지]

이후 생성된 4개의 곱셈 결과는 3개의 RCA10으로 구성된 2-stage adder tree를 통해 최종적으로 하나의 Dot Product 결과로 합산됩니다.

[30페이지]

accelerator의 schematic입니다.

입력으로 들어오는 4개의 데이터 쌍은 각각의 MUL4에 연결되어 4개의 8-bit product를 동시에 생성합니다.

앞선 MAC 구조처럼, 각 MUL4의 출력을 10bit RCA에 입력하기 위해 상위 비트를 VSS에 연결하여 10-bit로 zero extension했습니다.

첫 번째 stage에서는 두 개의 RCA10이 각각 두 product를 병렬로 더하고, 두 RCA의 결과는 두 번째 stage의 RCA10으로 전달됩니다.

마지막 RCA10에서 두 중간 결과를 다시 더하면서 최종 Dot Product 결과가 출력됩니다.

[31페이지]

4비트 MUL의 시뮬레이션 결과를 binary로 출력했습니다. weight에 해당하는 B를 2라는 값으로 고정하고, input에 해당하는 A를 순차적으로 변화시켜가면서 output P의 값을 확인해 보았습니다. 시뮬레이션 결과 두 input의 곱에 해당하는 값이 정상적으로 출력되는것을 확인할 수 있었습니다.

[32페이지]

MUL의 critical path에 해당하는 output bit를 확인하기 위해서 시뮬레이션한 모습입니다.

AND 블록과 ADDER 블록을 합쳐서 가장 많은 논리게이트를 지나야 하는 P7과 P6 비트 중 어떤 것이 가장 긴 delay time을 갖는지 시뮬레이션 해보았습니다. input을 변화시킴에 따라서 각 두 비트가 변화하는 시간을 확인하였는데, 확인 결과 P7이 가장 큰 0.385ns의 delay time을 가지는 것을 확인할 수 있었습니다.

[33페이지]

10bit RCA도 input을 3과 5라는 4bit input 2개를 각각 넣어서 8이 정상적으로 출력되는 것을 확인한 시뮬레이션 결과 사진입니다.

[34페이지]

MAC 시뮬레이션에 앞서서, 레지스터의 리셋 단자에 1V를 인가했을 때, 값을 0으로 초기화할 수 있는지 확인해 보았습니다. RSTB 단자에 clk 이전 짧은 시간동안 1V를 인가해서 처음 sum의 값이 모두 0으로 초기화되는 것을 확인하였습니다. 그 이후에 clk에 따라서 RCA에서 전달된 값들이 레지스터로 입력되어 정상적으로 출력되는 것을 확인할 수 있었습니다.

[35페이지 — MAC Simulation]

다음으로 MAC의 시뮬레이션 결과에 대해 설명드리겠습니다.

MAC의 input으로 3X5, 2X7, 15X15, 1X1 의 4가지 입력을 주어서, 각 계산을 clk의 상승엣지에 따라서 연산하도록 설정하였습니다.

시뮬레이션 결과 clock에 따라 출력이 15, 29, 254, 255 순서로 변화하는 것을 확인했습니다.

MAC은 각 clock마다 새로운 곱셈 결과를 그대로 출력하는 것이 아니라, 이전 register에 저장된 값에 현재 multiplier 결과를 더해서 다시 저장하는 누산 구조입니다.

앞선 레지스터 시뮬레이션처럼 Reset 단자에 짧은 시간 1V를 인가한 직후 accumulator는 0으로 초기화됩니다. 첫 번째 곱셈 결과가 15이므로 첫 번째 clock에서 0에 15를 더한 15가 register에 저장됩니다.

두 번째 clock에서는 이전 값 15가 feedback되고 새로운 곱셈 결과 14를 더해서 29가 됩니다. 세 번째에는 이전 값 29에 225를 더해서 254, 마지막에는 254에 1을 더해서 최종적으로 255가 저장됩니다.

따라서 최종 결과 255는 각각의 곱셈 결과인 15, 14, 225, 1을 순차적으로 누적한 결과입니다.

이처럼 하나의 multiplier와 RCA를 여러 clock에 걸쳐 재사용하는 것이 Sequential MAC 구조의 특징입니다. 총 네 개의 곱셈 결과를 누적하기 위해 네 번의 clock cycle이 필요하다는 것을 시뮬레이션을 통해서 시각적으로 확인할 수 있었습니다.

[36페이지]

다음으로 DOT product accelerator의 시뮬레이션 결과입니다.

accelerator는 모든 입력을 1111로 해서 최대값인 900의 연산 결과가 제대로 출력되는지 확인하였습니다.

시뮬레이션 결과 output 0이 한 클럭 사이클만에 900으로 출력되는 것을 확인할 수 있었습니다.

앞선 MAC 구조와 다르게 accelerator는 4개의 MUL을 병렬로 사용하기 때문에, 네번의 사이클이 필요했던 앞선 결과와 다르게 한 클럭의 짧은 시간만에 연산을 수행할 수 있었습니다. input이 들어간 시간과 900의 output이 출력된 delay time은 0.8142ns 로 확인했습니다.

[37페이지 — Layout Metal]

layout 결과에 앞서, layout에서 사용한 metal layer에 대해 설명드리겠습니다.

layout하기에 앞서서, 낮은 level의 metal은 local connection, 높은 metal은 상대적으로 장거리 연결용으로 사용하기로 계층화하였습니다.

이처럼 metal layer에 역할을 나누어 M1은 cell 내부 연결과 VDD/VSS처럼 local connection을 중심으로 두고, M2와 M3는 input인 A/B bus 및 local crossing에 사용하였습니다.

그리고 AND block에서 ADDER block으로 이동하는 비교적 긴 block 간의 interconnection은 M4를 사용하였습니다.

M5 이후부터는 최종 circuit을 구성할 때 top-level에서 routing이 막히거나 block 간 crossing이 복잡할 때 사용할 예정입니다.

[38페이지 — Multiplier Layout]

Multiplier layout에서는 두 기능을 분리해서, 왼쪽에는 AND block, 오른쪽에는 ADDER block을 배치했습니다. 이렇게 하면 입력 A와 B를 AND array에 먼저 분배하고, 생성된 partial product를 오른쪽 ADDER block으로 넘기는 방향으로 signal flow를 정리할 수 있어서 routing 구조도 더 명확해진다고 생각해서 이렇게 layout을 진행하였습니다.

[39페이지 — RCA10 Layout]

RCA10의 경우 carry가 FA0부터 FA9까지 순차적으로 전파되므로 carry propagation path가 주요 critical path가 된다는 단점이 있습니다.

따라서 layout에서는 인접 Full Adder의 COUT과 CIN을 최대한 짧게 연결하는 것이 중요합니다.

따라서 서로 연결되는 FA끼리 길게 수평으로 layout을 진행하였습니다.

[40페이지]

레지스터의 경우, 기존 DFF에 NMOS를 추가했기 때문에 추가적인 NMOS layout 및 배치 수정이 필요했습니다.

사진과 같이 RSTB 단자용 NMOS를 추가해서 수평으로 layout을 진행하였습니다.

[43페이지 — LVS Pin Mapping Optimization]

다음으로 LVS 과정에서 진행한 input mapping 최적화입니다.

Layout 과정에서 일부 Half Adder와 Full Adder 입력이 서로 교차하면서 metal crossing과 routing length가 증가하는 문제가 있었습니다.

이때 두 입력의 매핑을 바꾸면 layout이 훨씬 수월해질 수 있었는데, HA, FA의 두 입력은 논리적으로 대칭이기 때문에 입력 순서를 바꾸더라도 출력 기능은 동일합니다.

[44페이지]

따라서 layout에서 더 짧고 단순한 배선이 가능하도록 schematic을 해당 mapping에 맞춰 수정해서 layout을 진행

하였고,

이후 LVS를 다시 수행하여 schematic과 layout connectivity가 일치하는 것을 확인했습니다.

결과적으로 입력의 swap이 가능한 경우, layout에 유리한 방향으로 유동적으로 schematic을 수정해서 metal 배선의 길이를 줄일 수 있었고

이를 통해 schematic의 적절한 조정을 통해 layout을 더욱 최적화할 수 있다는 것을 배울 수 있었던 것 같습니다.

[45~46페이지]

이건 10bit RCA와 레지스터의 DRC 및 LVS 검증 과정을 진행한 사진입니다.

[50페이지 — Conclusion / MAC vs Dot Product]

각 결과들을 바탕으로 Sequential MAC과 Parallel Dot Product Accelerator를 비교해보았습니다.

결국 Sequential MAC과 Parallel Dot Product Accelerator는 동일한 수학적 연산을 서로 다른 하드웨어 구조로 수행합니다.

[51페이지]

MAC은 하나의 multiplier와 adder를 반복해서 사용하기 때문에 면적 측면에서는 유리하지만, register와 feedback으로 인해 여러 clock cycle이 필요하기 때문에 연산 시간이 더 오래 걸리게 됩니다.

반면 Parallel Dot Product Accelerator는 multiplier 4개와 adder tree를 사용하는 병렬 방식으로 면적은 증가하지만 하나의 clk 사이클에 4개 계산을 수행할 수 있습니다.

[52페이지]

위 표는 MAC의 clk이 가져야 하는 최소 시간과 accelerator의 연산 결과가 나올 때까지의 delay를 계산하는 식입니다. 이때, 설정할 수 있는 clk의 최소치는 각 구조의 인스턴스 별 critical path의 delay를 합친 값보다 길어야 합니다. 그렇기 때문에 clk을 아무리 최소치로 잡는다고 하더라도, 4클럭의 사이클이 필요한 MAC보다 parallel accelerator의 연산 속도가 빠르다는 결론을 얻을 수 있었습니다.

결론적으로 Sequential MAC은 하나의 multiplier와 adder를 여러 clock cycle에 걸쳐 재사용하기 때문에 연산 latency는 증가하지만, 필요한 하드웨어 자원을 줄일 수 있습니다. 따라서 칩 면적이나 하드웨어 자원이 제한되는 소형 디바이스나 edge device에서는 MAC 구조가 더 유리할 수 있다고 예상했습니다.

이러한 관계를 증명하기 위해서 최종적으로는 MAC과 Parallel Dot Product Accelerator의 전체적인 layout을 통해 두 구조의 실제 면적을 비교해보고, 면적과 연산 속도 간의 trade-off 관계를 수치적으로 비교하여 정리할 계획입니다.